Trappole d'esame — Dominio 1: Architettura agentica e orchestrazione

> **Come usare questo file.** Per ogni trappola trovi: la risposta sbagliata che l'esame ti propone (❌), perché è sbagliata, la risposta corretta (✅) e il segnale che ti aiuta a riconoscerla nella domanda (🔎). Dove la guida d'esame e la documentazione attuale (ottobre 2026) divergono, è segnalato con ⚠️ **Guida vs docs**: all'esame vale la guida.

Regole d'oro del dominio

1. **Il loop si controlla con `stop_reason`**, non con il testo, non con contatori, non con frasi del modello.
2. **I sottoagenti non ereditano nulla**: tutto ciò che serve va passato esplicitamente nel prompt.
3. **Il coordinatore è il punto di passaggio** di tutta la comunicazione e il primo indiziato quando manca copertura.
4. **Conformità al 100% = codice deterministico** (hook, gate), mai solo prompt.
5. **PreToolUse blocca, PostToolUse trasforma.**
6. **Contesto obsoleto → ripartire con un riassunto**, non riprendere né fare fork.

---

1. Loop agentici

Trappola 1 — Usare il tipo del primo blocco di contenuto per capire se il loop è finito

❌ `if response.content[0].type == "text": break`

Perché è sbagliato: una stessa risposta può contenere un blocco `text` (es. "Ora controllo l'ordine...") **seguito** da un blocco `tool_use`. La presenza di testo non significa che il modello abbia finito.

✅ Controlla `stop_reason`:

- `"tool_use"` → esegui gli strumenti, aggiungi i risultati, richiama l'API;
- `"end_turn"` → il modello ha finito, esci dal loop.

🔎 Qualsiasi opzione che ispeziona il *contenuto* della risposta per decidere la terminazione è sbagliata.

Trappola 2 — Un limite di iterazioni come meccanismo principale di arresto

❌ "Fermati dopo 10 cicli."

Perché è sbagliato: un numero arbitrario o interrompe lavoro utile (task che servono 12 passaggi) o lascia girare iterazioni inutili.

✅ `stop_reason` è il controllo principale; un tetto massimo di iterazioni va bene **solo come rete di sicurezza** contro loop infiniti.

🔎 Se l'opzione dice "come meccanismo principale" o "per decidere quando fermarsi", è la trappola. Se dice "in aggiunta, come salvaguardia", può essere accettabile.

Trappola 3 — Interpretare frasi del modello ("Ho finito", "Compito completato")

Perché è sbagliato: il linguaggio naturale è ambiguo e non deterministico; il modello può dire "ho finito" a metà lavoro o non dirlo mai.

✅ `stop_reason` è un segnale strutturato e inequivocabile.

Trappola 4 — Omettere i risultati degli strumenti dalla cronologia (o sostituirla con un riassunto) per risparmiare token

Perché è sbagliato: l'API Messages è **stateless**: non conserva nulla lato server tra una chiamata e l'altra. Se il `tool_result` non viene rimandato, il modello non ha niente di nuovo su cui ragionare e ripete la chiamata o inventa.

✅ Aggiungi ogni `tool_result` alla cronologia (come messaggio `user` che risponde al `tool_use_id` corrispondente) e invia l'intera conversazione. Se i risultati sono troppo grandi, **riduci ciò che lo strumento restituisce** (solo i campi rilevanti), non la cronologia.

---

2. Orchestrazione multi-agente (coordinatore + sottoagenti)

Trappola 5 — Dare la colpa ai sottoagenti per lacune di copertura

Esempio: il report sulle energie rinnovabili non parla di geotermico né di maree.

Perché è sbagliato: i sottoagenti ricercano solo ciò che viene loro assegnato. Se il coordinatore ha scomposto il tema in "solare" ed "eolico", nessun sottoagente poteva coprire il resto.

✅ Risali all'origine del difetto: la **scomposizione del task fatta dal coordinatore** era troppo ristretta. Correggi lì.

🔎 "Il report finale manca di X" → chiediti prima *chi ha deciso i sottoargomenti*.

Trappola 6 — Pensare che i sottoagenti condividano memoria con il coordinatore

Perché è sbagliato: un sottoagente **non eredita** la cronologia del coordinatore, il suo system prompt, né i risultati degli altri sottoagenti. Parte solo da ciò che il coordinatore scrive nel suo prompt.

✅ Passa esplicitamente tutto il contesto necessario (obiettivo, vincoli, risultati precedenti rilevanti).

Trappola 7 — Proporre comunicazione diretta tra sottoagenti "per efficienza"

Perché è sbagliato: i messaggi diretti aggirano il coordinatore e quindi perdi osservabilità, gestione coerente degli errori e controllo del flusso informativo.

✅ Tutta la comunicazione passa dal coordinatore (topologia hub-and-spoke).

⚠️ **Guida vs docs:** l'Agent SDK attuale (ottobre 2026) consente a un sottoagente di inviare messaggi ad agenti nominati con `SendMessage`. All'esame la risposta attesa resta "instrada tutto tramite il coordinatore".

Trappola 8 — Far passare ogni query per l'intera pipeline (magari con cache dei risultati inutilizzati)

Perché è sbagliato: invocare sottoagenti non necessari aggiunge latenza e chiamate API, che siano in cache o no. La cache non risolve il problema di fondo: il lavoro non serviva.

✅ Il coordinatore **analizza la query e invoca solo i sottoagenti necessari** (es. una semplice domanda fattuale → solo ricerca web).

---

3. Invocazione dei sottoagenti e passaggio di contesto

Trappola 9 — Assumere accesso automatico alla cronologia o agli output di altri sottoagenti

Stesso principio della trappola 6 ma in domande diverse: il contesto è **isolato**. Se il sottoagente B ha bisogno dell'output di A, il coordinatore deve inserirlo nel prompt di B.

Trappola 10 — Dare la colpa al prompt dell'agente di sintesi per le citazioni mancanti

Perché è sbagliato: l'agente di sintesi non può citare una fonte che non ha mai ricevuto. Se il coordinatore passa solo il testo dei risultati, URL, nomi dei documenti e numeri di pagina sono già persi.

✅ Il coordinatore passa i risultati come **dati strutturati** che tengono i metadati di provenienza separati dal contenuto, ad esempio:

```
{ "claim": "...", "source_url": "...", "document": "...", "page": 12 }
```

🔎 Il problema compare a valle (sintesi), ma la causa è a monte (passaggio di contesto).

Trappola 11 — Invocare in sequenza sottoagenti indipendenti

Perché è sbagliato: aggiunge latenza senza motivo.

✅ Il coordinatore emette **più chiamate allo strumento Task nella stessa risposta**, così i sottoagenti girano in parallelo. Usa la sequenza solo quando B dipende davvero dall'output di A.

Trappola 12 — Scrivere il prompt del sottoagente come procedura rigida passo per passo

Perché è sbagliato: una procedura fissa si rompe appena l'input è diverso da quello previsto e il sottoagente non può adattarsi.

✅ Indica **l'obiettivo e i criteri di qualità** del risultato atteso; lascia al sottoagente la scelta del *come*.

---

4. Applicazione del workflow e handoff

Trappola 13 — Prompt più forti o few-shot per errori di conformità ad alto rischio

Perché è sbagliato: migliorano la precisione, ma il modello resta probabilistico: il tasso di errore non arriva mai a zero.

✅ Per operazioni finanziarie, di sicurezza o di conformità usa **enforcement programmatico** (hook, gate prerequisiti, validazioni nel codice), che dà una garanzia deterministica.

Trappola 14 — Gestire una richiesta con più problemi uno alla volta

Esempio: il cliente chiede rimborso, cambio indirizzo e info su un ordine nello stesso messaggio.

Perché è sbagliato: il cliente riceve risposte frammentate, e i problemi trattati dopo rischiano di perdere il quadro d'insieme.

✅ **Scomponi** la richiesta in elementi distinti, analizzali **in parallelo** con un contesto condiviso (es. l'account del cliente), poi restituisci **una risposta unificata**.

Trappola 15 — Un classificatore di routing per risolvere problemi di conformità *dentro* un agente

Perché è sbagliato: il router decide *quale* agente gestisce la richiesta. Se l'errore avviene *durante* l'esecuzione dell'agente (es. rimborso emesso senza verificare l'identità), il routing non c'entra.

✅ Aggiungi un **gate prerequisito**: lo strumento a valle (`issue_refund`) resta bloccato finché il passaggio prerequisito (`verify_identity`) non è completato.

Trappola 16 — Riassunti di handoff incompleti

Perché è sbagliato: l'operatore umano **non vede la trascrizione** della conversazione: il riassunto è tutto ciò che ha.

✅ Il riassunto deve essere autosufficiente e contenere almeno: **ID cliente, sintesi della conversazione, analisi della causa, importo del rimborso (se applicabile), azioni raccomandate**.

---

5. Hook dell'Agent SDK

Hook	Quando gira	Vede	Serve a
`PreToolUse`	**prima** dell'esecuzione	la chiamata in uscita (nome + input)	bloccare / modificare / approvare la chiamata
`PostToolUse`	**dopo** l'esecuzione, prima che il modello legga il risultato	il risultato	normalizzare, filtrare, arricchire il risultato

Trappola 17 — Usare `PostToolUse` per bloccare azioni non conformi

Perché è sbagliato: quando l'hook scatta, lo strumento è già stato eseguito: il rimborso è già partito.

✅ Usa `PreToolUse` per bloccare prima che l'azione avvenga.

Trappola 18 — Istruzioni nel prompt per requisiti di conformità al 100%

Stesso principio della trappola 13: i prompt danno conformità **probabilistica**, gli hook **deterministica**.

🔎 Parole come "sempre", "mai", "100%", "obbligo normativo", "limite di policy" → la risposta è un hook o un controllo nel codice.

Trappola 19 — Lasciare al modello la normalizzazione di formati misti

Esempio: uno strumento restituisce date Unix, un altro ISO 8601, un terzo "25/10/2026".

Perché è sbagliato: il modello normalizza in modo incoerente.

✅ Un hook `PostToolUse` converte tutto in un formato unico prima che il risultato arrivi al modello.

Trappola 20 — Usare `PreToolUse` per ripulire i risultati

Perché è sbagliato: `PreToolUse` gira prima dell'esecuzione; il risultato non esiste ancora.

✅ La trasformazione dei risultati va in `PostToolUse`.

---

6. Strategie di scomposizione dei task

Trappola 21 — Modello più potente, finestra di contesto più ampia o prompt più dettagliato contro la "diluizione dell'attenzione"

Perché è sbagliato: la diluizione nasce dal **troppo materiale in un solo passaggio**. Le tre soluzioni proposte lasciano comunque tutto in un unico passaggio.

✅ Un **passaggio locale per ogni elemento** (es. per file) + un **passaggio di integrazione** trasversale.

Trappola 22 — Scomposizione dinamica/adattiva per un task con passaggi noti in anticipo

Perché è sbagliato: un piano adattivo rende durata e output dipendenti da ciò che emerge a ogni passo: meno prevedibile, più difficile da debuggare.

✅ Passaggi noti → **pipeline fissa** (prompt chaining): risultati coerenti e affidabili.

Trappola 23 — Pipeline fissa per un'indagine aperta

Perché è sbagliato: una pipeline fissa non può reagire a scoperte inattese.

✅ Ambito sconosciuto all'inizio → **scomposizione dinamica adattiva**.

🔎 Trappole 22-23 in coppia: "*passaggi noti / processo standard*" → fissa; "*indagine / causa sconosciuta / esplorare*" → adattiva.

Trappola 24 — Suddividere i file in batch senza un passaggio di integrazione

Perché è sbagliato: il batching riduce la diluizione *dentro* ogni lotto, ma i problemi *tra* lotti (flussi di dati, pattern incoerenti) restano invisibili.

✅ Aggiungi sempre un **passaggio di integrazione cross-file** dopo i batch.

---

7. Stato della sessione e ripresa

Situazione	Azione corretta
Contesto per lo più valido, pochi file cambiati	`--resume` + indicare **quali file** sono cambiati

Situazione	Azione corretta
Gran parte dei risultati degli strumenti obsoleti	**Nuova sessione** con riassunto strutturato iniettato
Voglio confrontare due approcci alternativi	`fork_session`
Proseguire la stessa linea di lavoro	`--resume` (non fork)

Trappola 25 — Riesplorare 50 file quando ne sono cambiati 3

Perché è sbagliato: spreco di tempo e token.

✅ Riprendi e indica all'agente i **3 file modificati** per un'analisi mirata; il riassunto precedente copre il resto.

Trappola 26 — Riprendere una sessione i cui risultati degli strumenti sono quasi tutti obsoleti

Perché è sbagliato: la cronologia ripresa contiene il vecchio contenuto dei file; l'agente ragiona su dati superati e dà consigli contraddittori.

✅ **Riparti da zero** con un riassunto strutturato. (Se invece la maggior parte del contesto è ancora valida → vedi trappola 25.)

Trappola 27 — Usare `fork_session` per portare avanti una sola linea di indagine

Perché è sbagliato: il fork crea una nuova sessione da una copia della cronologia lasciando intatta l'originale: il lavoro finisce diviso tra due sessioni.

✅ `--resume` per continuare; `fork_session` solo per **esplorare approcci divergenti** dallo stesso punto.

Trappola 28 — Usare `fork_session` per risolvere un contesto obsoleto

Perché è sbagliato: il fork eredita la cronologia, quindi eredita anche i risultati obsoleti.

✅ Nuova sessione + riassunto iniettato.